

SPRINT 3 – BRIEF 1

ClickTrack

ClickTrack *est une API Spring MVC qui permet aux publishers de créer des liens de tracking et de suivre leurs clicks*

Contexte

Les publishers (influenceurs, YouTubeurs, blogueurs) partagent chaque jour des liens vers les offres qu'ils promeuvent, sans savoir lesquels intéressent vraiment leur audience. Une agence de marketing digital souhaite leur proposer **ClickTrack** : un service qui génère des liens de tracking et compte chaque click. À chaque ouverture(*click*) d'un lien, le click est enregistré et les informations du lien sont affichées. Le publisher suit ainsi les clicks par lien et par offre.

Spécifications fonctionnelles

User stories

- En tant que publisher, je crée un lien de tracking vers une URL de destination, rattaché à une offre.
- En tant que publisher, je liste mes liens avec leur nombre de clicks.
- En tant que publisher, je liste mes offres avec leur nombre de clicks.

Règles de gestion

- Une offre peut avoir plusieurs liens : ses clicks sont la somme des clicks de ses liens.
- À chaque ouverture d'un lien de tracking, le click est compté puis les informations du lien sont affichées : URL de redirection, nom et prénom du publisher, offre et nombre de clicks.
- Aucun click ne doit être perdu lors de clics simultanés.

Spécifications techniques

Entités

- **Publisher** : id, nom, prenom, email, categorie (enum).
- **Lien** : id, offre, urlDestination, clicks, publisherId.

Endpoints

Méthode	URL	Réponse
POST	**/api/publishers/{id}/liens**	201
GET	**/api/publishers**	200
GET	**/api/publishers/{id}/liens**	200
GET	**/api/publishers/{id}/offres**	200
GET	**/t/{id}**	200 / 404

Exemple de réponse de **GET /t/{id}** (après incrémentation du clic) :

Body Cookies Headers 5 Test Results 1/2 200 OK

{ } JSON Preview Visualize


```
{
    "urlRedirection": "https://www.exemple.com/casque-audio",
    "publisher": "Sara Alami",
    "offre": "Casque audio X200",
    "clicks": 11
}
```


Exemple de réponse de **/api/publishers**:

Body Cookies Headers 5 Test Results 2/2 200 OK 31 ms 489 B

{ } JSON Preview Visualize


```
[
    {
        "id": 1,
        "nom": "Alami",
        "prenom": "Sara",
        "email": "sara.alami@example.com",
        "categorie": "INFLUENCEUR"
    },
    {
        "id": 2,
        "nom": "Bennani",
        "prenom": "Youssef",
        "email": "youssef.bennani@example.com",
        "categorie": "YOUTUBEUR"
    },
    {
        "id": 3,
        "nom": "El Idrissi",
        "prenom": "Imane",
        "email": "imane.elidrissi@example.com",
        "categorie": "BLOGUEUR"
    }
```


Exemple de réponse de **/api/publishers/{id}/offres**:

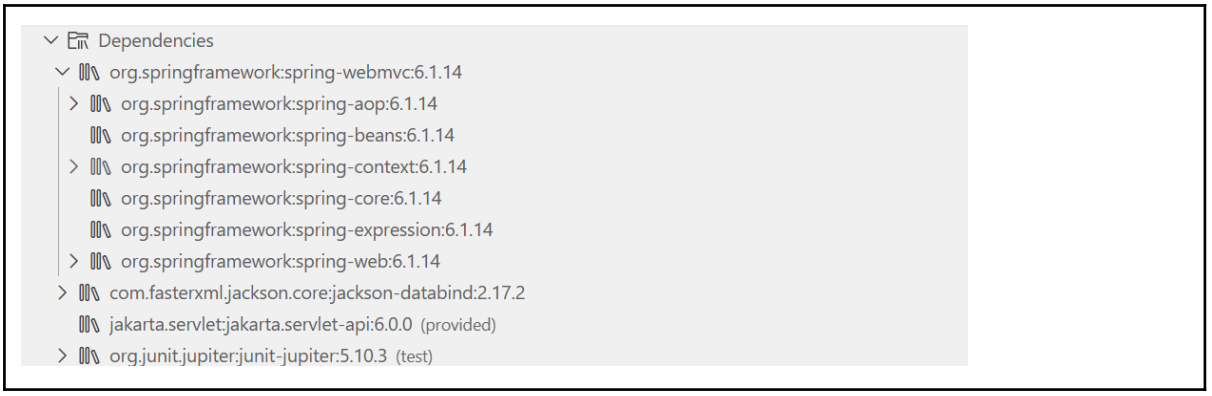
Body Cookies Headers 5 Test Results 2/2 200 OK

{ } JSON Preview Visualize


```
[
    {
        "offre": "Casque audio X200",
        "nombreLiens": 1,
        "clicks": 10
    },
    {
        "offre": "Montre connectée Z5",
        "nombreLiens": 1,
        "clicks": 2
    }
]
```

Exigences

- Architecture en couches : controller, service, repository.
- Injection par constructeur/setter, dépendances vers des interfaces.
- **Stockage en mémoire** : les repositories conservent les données dans des **ArrayList**.
- Java 17+, Maven, WAR, Tomcat 10.1+.
- L'équipe a choisi **Spring Core et Spring MVC (bonus: spring ORM - hibernate)**:



- **Gestion de la concurrence :** plusieurs visiteurs peuvent cliquer en même temps, l'incrémentation des clicks, l'ajout dans les listes est protégé, afin qu'aucun click ni aucune donnée ne soit perdu.

Les 3 configurations

Le projet est livré avec une seule configuration: **configuration XML**.

Config	Déclaration des beans	Démarrage de Spring MVC
config-xml	<bean> et <constructor-arg> dans applicationContext.xml, aucune annotation de stéréotype	web.xml + dispatcher-servlet.xml
config-java	Méthodes @Bean dans des classes @Configuration, aucun scan	AbstractAnnotationConfigDispatcherServletInitializer
config-annotation	@Service, @Repository, @RestController, détection par @ComponentScan	AbstractAnnotationConfigDispatcherServletInitializer

Les annotations de mapping (@RequestMapping, @GetMapping, @PathVariable…) sont autorisées.

Bonus – Redirection vers la page de destination et Initialisation des données

Aujourd'hui, **GET /t/{id}** compte le click puis affiche les informations du lien. Dans un vrai service de tracking, le visiteur doit arriver directement sur la page de destination.

- Modifier **GET /t/{id}** pour qu'il compte le click, puis redirige le visiteur vers l'URL de destination

Initialisation des données : un bean DataInitializer crée les publishers de test au démarrage de l'application, dans sa méthode d'initialisation (phase *init* du cycle de vie du **bean**)